5 ：React・フロントエンド — の み て （ ）

> React（リアクト） アプリの のコンポーネント `KosuList.tsx` 215 を から まで1 ずつ
>
> （コンポーネント）
> データに じて を で き える
>
> `useState` `useEffect` `useCallback` `useRef` `props`
> `JSX` アプリの
> コードだけ
> `KosuList`
>
> のように えること `useEffect`

この で ぶこと

- Reactとは か
- コンポーネント `KosuList` `Pagination` `Loading` `TableContainer`
- JSX
- props（プロップス） `Pagination`
- state（ステート）と `useState`
- `useEffect` `useEffect`
- `useRef`
- `useCallback` `fetchData`

- `KosuList.tsx`　215　を1　ずつ

- `Pagination.tsx`　71　を1　ずつ
- `Loading.tsx`・`TableContainer.tsx`
- `axios.ts`　8
- `index.tsx` のルーティング　`Routes`　`Route`　`:id`　`Link`　`useNavigate`
- トラブルシューティング・

0.　　：Reactが　　した

`useState`

`useEffect`

0.1 Reactがなかった　　の

```
// ※　　JavaScript　1　　す
//　　tr
const tr = document.createElement("tr");
//　　td
const td = document.createElement("td");
//
td.textContent = "2025-06-01";
//　　し
tr.appendChild(td);
//　　し
document.getElementById("tbody").appendChild(tr);
```

「どのDOMを、どこに、どう　し　むか」を　　で

0.2 Reactの　　—「データを　えたら、　　は　　に　り　す」

> **Reactの　　：　　のデータなら、　　はこうあるべき　　*設計図*
> 　　い　と　しい　　を　べて、　わった　だけを
> 　で　き　える**

（　　）を　せば、シェフ（React）が　ある　　で　　に　り　け　し
てくれる

> :　　（せんげんてき／declarative）

0.3　　DOM —「　　きで　べてから　　する」

DOM（かそうディーオー
エム）

> :　　DOM（Virtual DOM）
>
> 　　う　　だけ

の
き（　　DOM=　い）

0.4　アプリにおけるReactの　　づけ

		アプリの
HTML（JSX）		`.tsx`　`return ( ... )`
CSS		`.module.css`　`global.css`
React + TypeScript		`.tsx`　　`useState`

という　え

の「　」

mponent：　・　）
の　が1つの

```
い　　←　き

す　　←　く　い

　　←　さ　　い　せ
　　←　さ　　い　せ
```

```
// Login   う                  。React.FC   「React                  」   う
const Login: React.FC = () => {
  // ...              す        ...
  // こ      が  く      JSX      す
  return (
    <div>...</div>
  );
};

// こ            か   import   き    う
export default Login;
```

- `const Login = () => { ... }` アロー
- `: React.FC` F C

- `return ( <div>...</div> )` `( )`

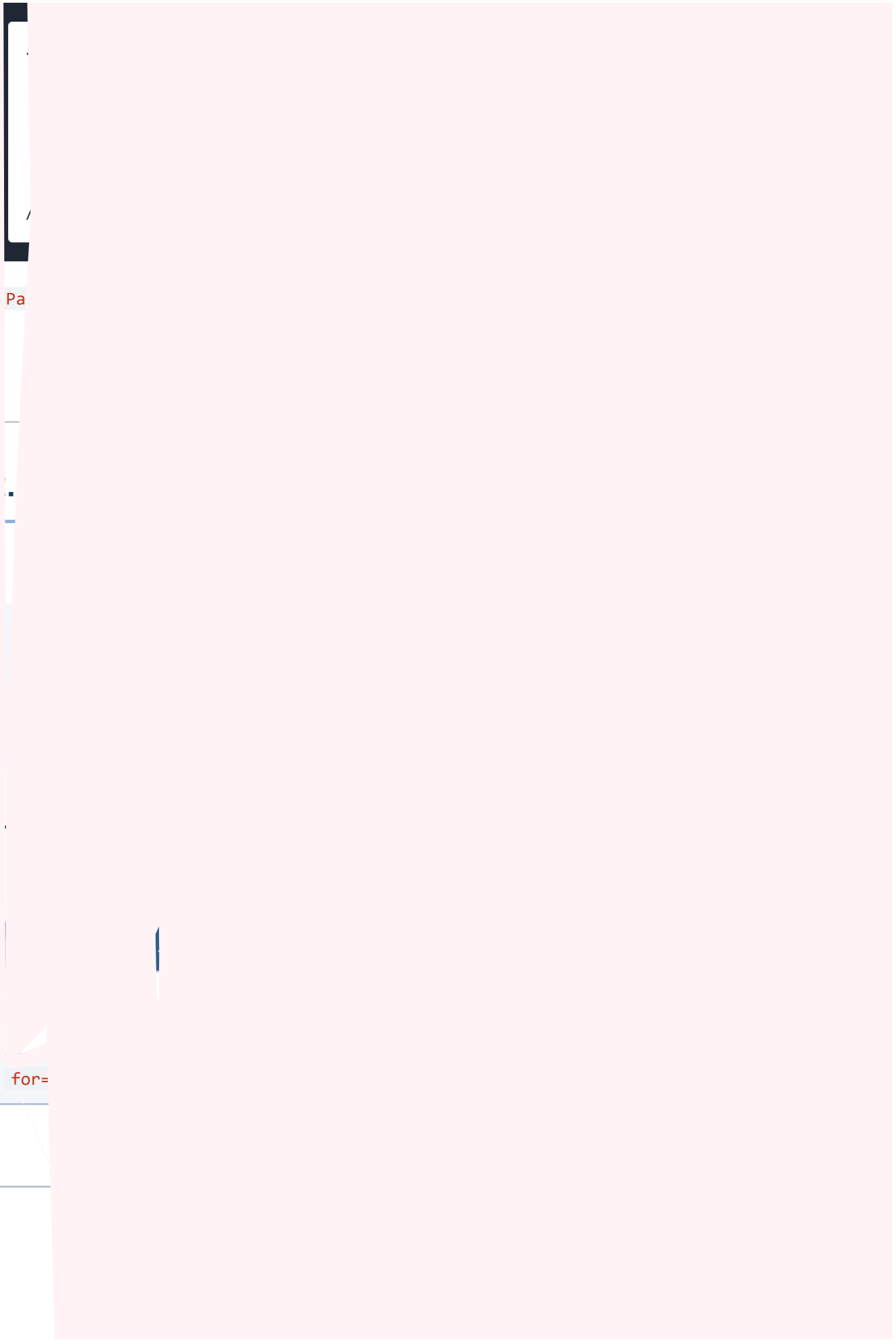
- `export default Login` **default export** `import Login from "./Login"`

> **: React.FC（エフシー）** `FunctionComponent`

> ⚠️ **コンポーネント は ず まり** `Login` `KosuList` `Pagination`
> `login`
> `<login>`

1.3 を「 う」とは ―タグのように く

HTMLタグのように `KosuList.tsx`
`Pagination`



```
{/* for        <  htmlFor */}
<label htmlFor="numberInput">          </label>
{/*
<input
```

2.2 `{ }` —

ꓭ9ꓶꓶOꓶ=9{\\} ꓭ9ꓶꓶOꓶ=9\\

μ KosuList.tsx

```
```

> **なぜ `&&` で　できる？**　`A && B`
>
> 　　`errorMessage`
>
> 　`<div>`

(B)　　`条件 ? A : B` —「2　」

`KosuList.tsx`

```
{data.length === 0 ? (
  //　　が0　　、こ
  <p>No data found.</p>
) : (
  // 1　　、
  <TableContainer>...</TableContainer>
)}
```

`{　　?　　き :　　き}`

2.4 リストの　　：`map`

`map`　　`KosuList.tsx`

```
{data.map((item) => (
  <tr key={item.id}>
    <td>{item.work_day2}</td>
    ...
  </tr>
))}
```

`data`　　`item`　`<tr>`
`<tr>`

> ⚠️ **`key` を　れない**　`map`　　`key={　　}`　　`item.id`

2.5 フラグメント `<> ... </>`

`<div>` フラグメント `<> </>` `KosuList.tsx`

```
return (
  {/*  「 え い 」。 div い */}
  <>
    <div className={styles["kosu-list-wrapper"]}>
      ...
    </div>
  </>
);
```

: フラグメント（Fragment） `<>` `</>`
`<div>`

---

3. props — から へ す

3.1 propsとは —「 に す 」

: props（プロップス／properties の : ・ ）
に す `<img src="...">` `src`
`<Pagination currentPage={1} />`

の 扱 に く
`< ={25} =" " />`

3.2 propsは「　から　」へ。　　しない

　　propsは　→　の

```
  KosuList
  |  props     す   currentPage=1
  ▼
  Pagination    ←   け              う。   き   え   し   い
```

> なぜ　　なの？
>
> 　　から　された　　を　ぶ
>
> 　　`setCurrentPage`

3.3 propsの　け　り　（　　つき）

　　どんな props が　るか

`interface`　　`Pagination.tsx`

```tsx
// こ　　が　け　　props　「　　」
interface PaginationProps {
  //
  currentPage: number;
  //
  totalPages: number;
  //　　え　「　　」　　け
  setCurrentPage: (page: number) => void;
  //　　?　「　　」
  buttonColor?: string;
  //
  hoverColor?: string;
}
```

- `currentPage: number`　　`currentPage`
- `setCurrentPage: (page: number) => void`

　　`setCurrentPage`



4. state と useState ―　　が　える

4.1 stateとは ―「　　する、　えておく　」

> 　：state（ステート：　　）　　えておき、　　したら　　を　り
> す

	props	state
		その　　が　　される

4.2 useState の

　　`useState`

> 　：フック（hook）　`use`　　`useState`　`useEffect`
> `useRef`　`useCallback`
> 　　ずコンポーネント　　の　　（トップレベル）で
> ぶ

`KosuList.tsx`

> ▼ このコードがやること（　に　　で）：
> 　　`currentPage`　　`setCurrentPage(　しい　)`

```
const [currentPage, setCurrentPage] = useState<number>(1);
```

- `useState<number>(1)` 1
- `[currentPage, setCurrentPage]` `useState` 2つの が った
  - `currentPage` の 1
  - `setCurrentPage` を する の `setCurrentPage(2)` が で り されます

> ⚠️ stateは き えてはいけない `currentPage = 2;` ず `setCurrentPage(2)` のように を います

4.3 「 を ぶと される」のしくみ

```
① setCurrentPage(2)
      ▼
② Reactが「currentPage が 2 」
      ▼
③ KosuList が う か さ
      ▼
④ currentPage が 2 JSXが
      ▼
⑤ DOM し、 け
```

> な づき: `KosuList` だけ されるのではなく、state が わるたびに も から されます

4.4　アプリで　っているstate　　（KosuList）

`KosuList.tsx`

```
//　　す　　。　　[]
const [data, setData] = useState<Kosu[]>([]);
//　　か。　　true
const [loading, setLoading] = useState<boolean>(true);
//　　。　　null　　し
const [error, setError] = useState<string | null>(null);
//　　す　か。　　false
const [searchByMonth, setSearchByMonth] = useState<boolean>(false);
//　　。　　1
const [currentPage, setCurrentPage] = useState<number>(1);
//　　。　　0
const [totalPages, setTotalPages] = useState<number>(0);
```

`Kosu[]`　`boolean`　`string | null`　`number`

`string | null`　　または　　ユニオン

---

5. useE　ect ― タイミングで　　を　らせる

5.1 useE　ectとは ―「　　の“ついでに”やる　　」

> : useE　ect（ユーズエフェクト）／　　（side e　ect）
> サーバーから　　でデータを　る　ページタイトルを　える　タイマーを　　ける　イベントを　　する

`api.get(...)`　　のたびに　　が　り

`useEffect`

5.2 useE ect の と「 」

```
useEffect(() => {
  // こ こ
// ← こ  [] が「  」
}, [  す  1,  す  2]);
```

-
- （dependency array） わったとき

> : （いそんはいれつ） useEffect [...]
>
> - [] の1 だけ
> - [a, b] a b
> - かない の に

まし のセット [currentPage]

5.3 アプリの2つの useE ect（ ）

KosuList.tsx useEffect

に たらリセットする e ect

```
useEffect(() => {
  //
  searchDayRef.current = "";
  // ...  、  、  1  す...
// URL  が  き =こ  き
}, [location.pathname]);
```

データを りに く e ect

```
useEffect(() => {
  //         か
  fetchData(currentPage, searchDayRef.current, searchByMonth);
//                            が
}, [currentPage, fetchData, searchByMonth]);
```

`setCurrentPage`

5.4 クリーンアップ（　　け）

`useEffect`　　**を return**　　**け（クリーンアップ）**

`TableContainer.tsx`

```
useEffect(() => {
  //
  window.addEventListener("resize", updateDimensions);
  // ←       け
  return () => {
    //
    window.removeEventListener("resize", updateDimensions);
  };
}, [...]);
```

> **なぜ　　けが　るの？**
>
> 　　`Loading.tsx`　`clearTimeout`　`TableContainer.tsx`　`removeEventListener`

---

6. useRef —　　を　こさない

6.1 useRefとは —「　えるけど　　は　　しない　」

> 　　：useRef（ユーズレフ／reference：　　）　　その　　を　き
> えても　　は　きない　　`.current`

	useState	useRef
		きない
	`[　,　　]`	`ref.current`

6.2　　A：　　な　を　える

`KosuList.tsx`

```
//　　す　　え　　。
const searchDayRef = useRef<string>("");
// ...
//　　が　　し
onChange={(e) => { searchDayRef.current = e.target.value; }}
```

> **なぜ state ではなく ref？**　　`setState`
> 　　っている
> は　　`useRef`　　`searchDayRef.current`

6.3 B：DOM を つかむ

`KosuList.tsx` のDOM

```
// <div> か
const searchBarRef = useRef<HTMLDivElement>(null);
// <h1> し か
const headerRef = useRef<HTMLHeadingElement>(null);
// ...
// ref け
<div ref={searchBarRef} classNa
```

`ref={searchBarRef}` `searchBarRef.current` のDOM

`TableContainer.tsx` `searchBarRef.current?.offsetHeight`

> ：マウント／アンマウント マウント アンマウント `ref` `null`

---

7. useCallback — を い す

7.1 ： のたびに が「 り される」

7.2 useCallback —「　　が　じなら　じ　　を　い　す」

> 　　：useCallback（ユーズコールバック）　　メモ　（memoization：　　して　いす）　　と　じ　　オブジェクトを　す　　`useEffect`



```
// ① React  、  う4
import React, { useState, useEffect, useCallback, useRef } from "react";
// ②                §12
import api from "../api/axios";
// ③              う    axios
import axios from "axios";
// ④              URL
import { Link, useNavigate, useLocation } from "react-router-dom";
// ⑤          す          §11
import TableContainer from "../Components/TableContainer";
// ⑥                  §9
import Pagination from "../Components/Pagination";
// ⑦                §10
import Loading from "../Components/Loading";
// ⑧ こ            CSS  CSS Modules
import styles from "../styles/KosuPage/KosuList.module.css";
```

- `import React, { useState, useEffect, useCallback, useRef } from "react"` `react` `React` きで `{ }`
- `import api from "../api/axios"` `api` `api.get(...)` `../`
- `import axios from "axios"` `axios` `axios.isAxiosError(err)` エラーがaxios か する `api`
- `import { Link, useNavigate, useLocation } from "react-router-dom"` `Link` `useNavigate` `useLocation`
- ～ `TableContainer` `Pagination` `Loading`
- `import styles from "..."` `styles["kosu-list-wrapper"]`

8.2 データの　　interface Kosu（10〜17　）

> **▼ このブロックがやること:**
>
> 　　`name`

```
//　　1
interface Kosu {
  //　　　　ID
  id: number;
  //
  employee_no3: number;
  //
  name: string;
  //　　"2025-06-01"　　う
  work_day2: string;
  //　　く　　。"1"〜"6"
  tyoku2: string;
  //　　true=OK / false=NG
  judgement: boolean;
}
```

- `interface Kosu { ... }`　　`Kosu`

- 　　`:　;`　`id`　　`name`　　`judgement`
- 　　`useState<Kosu[]>([])`　　`Kosu`

> **なぜ　を　するの？**
>
> 　　`item.namae`

8.3　　formatTyoku（19〜29　）

> **▼ このブロックがやること:**

```
//                      す
const formatTyoku = (value: string | number): string => {
  // value             し        け
  switch (Number(value)) {
    // 1      「1  」
    case 1: return "1  ";
    // 2      「2  」
    case 2: return "2  ";
    // 3      「3  」
    case 3: return "3  ";
    // 4      「      じ  う    う  」
    case 4: return "    ";
    // 5      「  1  」
    case 5: return "  1  ";
    // 6      「  2  」
    case 6: return "  2  ";
    //           け
    default: return "";
  }
};
```

- `(value: string | number): string`

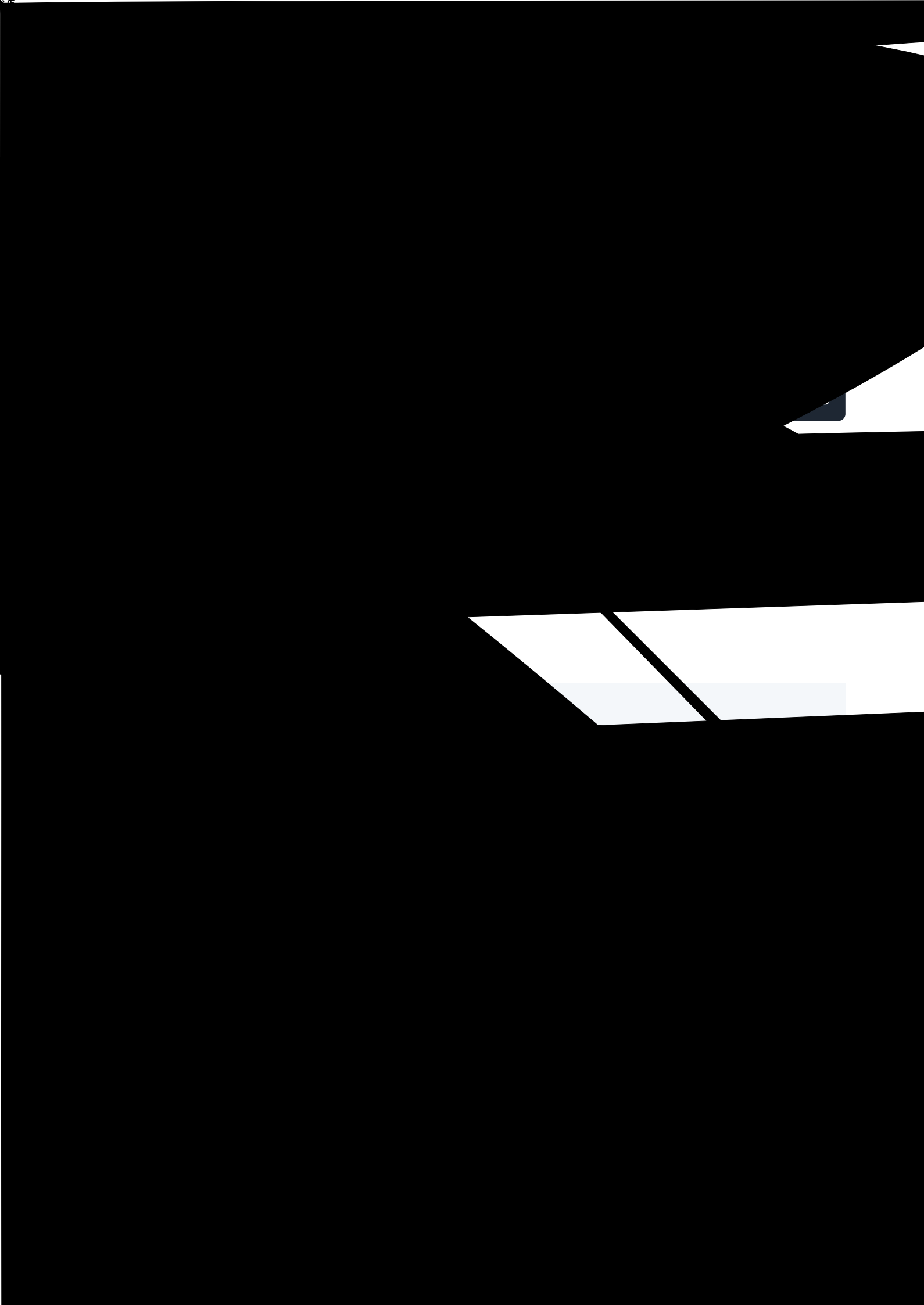
- `switch (Number(value))` `value` `Number(...)` `switch`
  `"1"` `1` `case 1`
- `case 1: return "1  ";`
- `default: return "";`

> **なぜコンポーネントの　に　くの？**
>
> **のたびに　り　され**
>
> **ず**

`{formatTyoku(item.tyoku2)}` `"1"` `"1  "`

8.4　　getDayOfWeek（31〜35　）

> **▼ このブロックがやること:**



- `const [totalPages, setTotalPages] = useState<number>(0)`
  `0`
- `const location = useLocation()`
  `location.pathname`
- `const navigate = useNavigate()` `navigate("/login")`
  プログラムから を

8.6 DOM のref（48〜49 ）

```
//  <div>  か 。  null
const searchBarRef = useRef<HTMLDivElement>(null);
//  し <h1>  か 。  null
const headerRef = useRef<HTMLHeadingElement>(null);
```

- `useRef<HTMLDivElement>(null)` DOM をつかむ
  `HTMLDivElement`
- `null`

- `<div ref={searchBarRef}>` `<h1 ref={headerRef}>`
  `TableContainer`

8.7 fetchData —データ （51〜84 ）

の `useCallback`

> ▼ このブロックがやること: `/api/kosu_list/`

```
// ① useCallback        し
const fetchData = useCallback(async (
  //     1      す
  page: number,
  //     2      す                    し
  day: string,
  //     3  true=       / false=
  mode: boolean
) => {
  // ②
  setLoading(true);
  // ③                   す  か  し    い     try
  try {
    // ④           GET
    const response = await api.get("/api/kosu_list/", {
      // ⑤ URL    け
      params: {
        //
        page: page,
        // ⑥ day が      け    、          し
        ...(day && {
          //
          day: day,
          //              "month"、          "day"
          mode: mode ? "month" : "day",
          //
          filter: "true",
        })
      }
    });

    // ⑦     か                    す    け
    const results = response.data.results || [];
    // ⑧ 1      あ            け    20
    const pageSize = response.data.page_size || 20;
    // ⑨         state        →
    setData(results);
    // ⑩                 し
    setTotalPages(Math.ceil(response.data.count / pageSize));
  // ⑪     が    し    こ こ
  } catch (err) {
    // ⑫       がaxios                  か
    if (axios.isAxiosError(err)) {
      // ⑬ 401=          →
      if (err.response?.status === 401) navigate("/login");
      // ⑭ 403=      し→
      else if (err.response?.status === 403) navigate("/");
      // ⑮
```

```
      else setError(err.response?.data.message);
    // ⑯ axios        い    せ
    } else {
      // ⑰
      setError("          が    し   し   。IT          し   く   さい。");
    }
  // ⑱                  ず
  } finally {
    // ⑲                      す            す
    setLoading(false);
  }
// ⑳          navigate が       け      い   す
}, [navigate]);
```

- `const fetchData = useCallback(async (...) => {` `useCallback` `async` `await`
- `page` `day` `mode`
- `setLoading(true)` `loading` `true` ヘルメットのローディング
- `try {` `try/catch`
- `const response = await api.get("/api/kosu_list/", {...})` `api` `/api/kosu_list/` GETリクエスト `await` が るまで ち `response`

> : GET（ゲット）リクエスト
> `api.get(...)`
> `api.post(...)` `Login.tsx`

- `params: { ... }` クエリパラメータ `?page=2&day=...`
- `...(day && { ... })` `day` `{ day, mode, filter }` スプレッド（`...`）で して params に `day` `day && {...}`

> **：スプレッド　（`...`）**
> `...{a:1, b:2}`　`a:1, b:2`

- `mode: mode ? "month" : "day"`
`"month"` `"day"`
- `const results = response.data.results || []`　`response.data`
`results`　`|| []`

> **：response.data**　`response`
> `response.data`

- `const pageSize = response.data.page_size || 20`
- `setData(results)`　`data`
- `setTotalPages(Math.ceil(response.data.count / pageSize))`
`count`　`pageSize`　`Math.ceil`
3ページ

> **：Math.ceil（マスドット・シール）**
> `Math.ceil(2.1)`　`Math.ceil(2.9)`　`3`　`ceil`

- `catch (err) {`　`err`
- `if (axios.isAxiosError(err))`　`import axios`
- `if (err.response?.status === 401) navigate("/login")`　401
（　＝ログインしていない）　`navigate`

- `err.response?.status` `?.` オプショナルチェーン `response` `status` `undefined`

> : HTTPステータスコード 200 401 403 404 500

- `else if (err.response?.status === 403) navigate("/")` 403（ なし）
- `else setError(err.response?.data.message)` `data.message` `error`
- `else { setError("不明な...") }`
- `finally {` ず
- `setLoading(false)`
- `}, [navigate])` `useCallback` `navigate` `fetchData` じ として い され

8.8 useE ect リセット（86〜92 ）

> ▼ このブロックがやること: `/kosu-list`

```
useEffect(() => {
  // ①
  searchDayRef.current = "";
  // ②          DOM    か
  const input = document.getElementById("search-day-input") as HTMLInputElement;
  // ③
  if (input) input.value = "";
  // ④                 false
  setSearchByMonth(false);
  // ⑤          1    す
  setCurrentPage(1);
// ⑥ URL       が
}, [location.pathname]);
```

- `searchDayRef.current = ""`
- `const input = document.getElementById("search-day-input") as HTMLInputElement` `as HTMLInputElement`
- `if (input) input.value = ""` `defaultValue`
- `setSearchByMonth(false)`
- `setCurrentPage(1)`
- `}, [location.pathname])` `location.pathname` `/kosu-list`

> **なぜリセットが　？**

8.9 useE ect　データ　（94〜96　）

> **▼ このブロックがやること:** `fetchData`

```
useEffect(() => {
  //
  fetchData(currentPage, searchDayRef.current, searchByMonth);
// こ     が
}, [currentPage, fetchData, searchByMonth]);
```

- `fetchData(currentPage, searchDayRef.current, searchByMonth)`
- `[currentPage, fetchData, searchByMonth]`
  - `currentPage` しいページを
  - `searchByMonth` り し
  - `fetchData` `useCallback`

> の : `Pagination` `setCurrentPage(2)` `currentPage` この e ectが `fetchData(2, ...)` `setData` ボタン1つの で、これだけの が で きています

8.10 handleSearch — ボタンの （98〜102 ）

> ▼ このブロックがやること:

```
//                 true
const handleSearch = (isMonthSearch: boolean) => {
  // ①
  setSearchByMonth(isMonthSearch);
  // ②          1    す
  setCurrentPage(1);
  // ③ 1
  fetchData(1, searchDayRef.current, isMonthSearch);
};
```

- **`setSearchByMonth(isMonthSearch)`** `true` `false`

- `setCurrentPage(1)`
- `fetchData(1, searchDayRef.current, isMonthSearch)`

> なぜ と が あるの？ `setCurrentPage(1)`
> に

8.11 リターン（104〜105 ）

> ▼このコードがやること:

```
// ①
if (loading) return <div><Loading isLoading={loading} /></div>;
// ②
if (error) return <div>Error: {error}</div>;
```

- `if (loading) return <div><Loading isLoading={loading} /></div>` `loading` `true` `Loading` `isLoading={loading}` propsの け し `Loading`
- `if (error) return <div>Error: {error}</div>` `error`

> : リターン（early return） `return`

8.12 JSX — の とnav（107〜119 ）

```
return (
  {/* ①                    div        い    */}
  <>
    {/* ②                                  */}
    <div className={styles["kosu-list-wrapper"]}>
      <h1
        {/* ③ こ   <h1>  headerRef    か       さ          */}
        ref={headerRef}
        className={styles["h1-collar"]}
      >
        {/* ④     し          */}

      </h1>

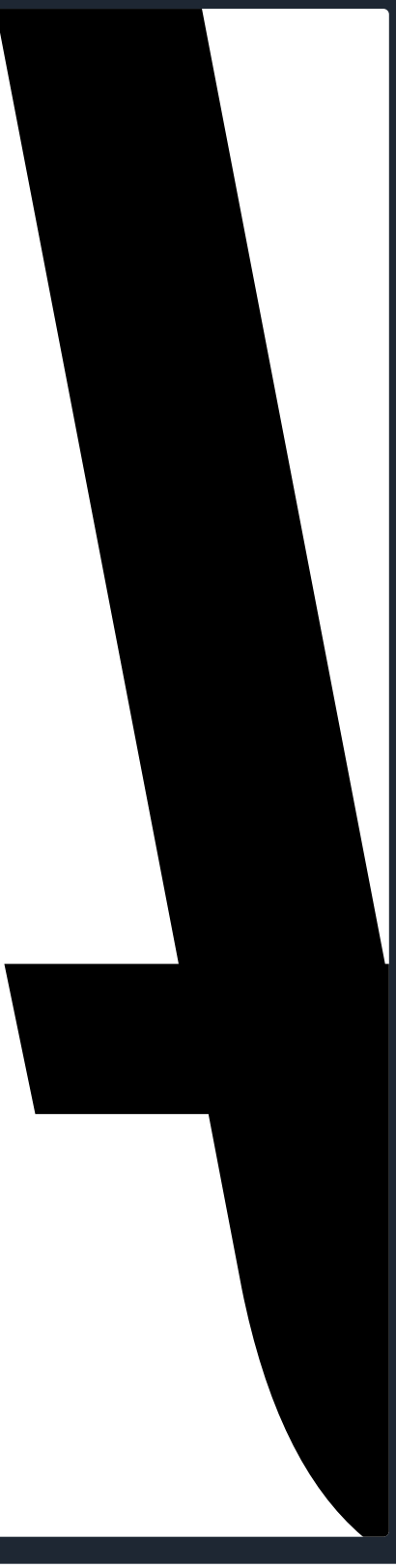
      {/* ⑤                     */}
      <nav className={styles["kosu-nav"]}>
        {/* ⑥                          */}
        <Link to="/kosu-menu">    MENU</Link>
      </nav>
```

- `<>`
- `<div className={styles["kosu-list-wrapper"]}>` `styles["..."]`
- `ref={headerRef}` `headerRef` `<h1>` `headerRef.current` `TableContainer`
- `工数履歴`
- `<nav className={styles["kosu-nav"]}>`
- `<Link to="/kosu-menu">工数MENU</Link>` `Link` ページ を み みせず `to="..."`

8.13 JSX — バー（121〜148 ）

> ▼ このブロックがやること:

```
<div
  {/* ①           <div>  ref    が        さ          */}
  ref={searchBarRef}
  className={styles["search-bar"]}
>
  {/* ②                              */}
  <label htmlFor="search-day-input"></label>
  <input
    {/* ③          ID  label          が       */}
    id
```



- `<input id="search-day-input" type="date">`

- `defaultValue={searchDayRef.current}` `value` ではなく `defaultValue`
  `value`
  コンポーネント

> : ／ コンポーネント state で
> `Login.tsx` `value={employee_no}` DOMに せる
> `defaultValue`

- `onChange={(e) => { searchDayRef.current = e.target.value; }}`
  `e.target.value` refに

- `placeholder="日付を選択"`

- `<div className={styles["button-group"]}>`

- `onClick={() => handleSearch(true)}`
  `handleSearch(true)`

> ⚠️ `onClick={handleSearch}` `onClick={() => handleSearch(true)}`
> んで `onClick={handleSearch(true)}`

- `onClick={() => handleSearch(false)}` `handleSearch(false)`

- `className="light_blue_button"` の
  `styles[...]`

8.14 JSX — と （150〜200 ）

> ▼ このブロックがやること:

`TableContainer` `map`

```
{/* ①       0      ↓ */}
{data.length === 0 ? (
  <p>No data found.</p>
{/* ② 1          ↓ */}
) : (
  <TableContainer
    {/* ③           ref    す    さ         */}
    searchBarRef={searchBarRef}
    {/*        し  ref    す */}
    headerRef={headerRef}
  >
    <table>
      <thead>
        <tr>
          {/* ④              */}
          <th className={styles["th-collar"]}>      </th>
          <th className={styles["th-collar"]}>  </th>
          <th className={styles["th-collar"]}>      </th>
          <th className={styles["th-collar"]}>    </th>
          <th className={styles["th-collar"]}>    </th>
        </tr>
      </thead>
      <tbody>
        {/* ⑤             1  ず               */}
        {data.map((item) => (
          {/* ⑥ key        id       */}
          <tr key={item.id}>
            {/* ⑦              */}
            <td>{item.work_day2} ({getDayOfWeek(item.work_day2)})</td>
            {/* ⑧                        し       */}
            <td>{formatTyoku(item.tyoku2)}</td>
            <td
              className={
                {/* ⑨                        */}
                item.judgement
                  {/*    true→OK           */}
                  ? styles["status-ok"]
                  {/*    false→NG           */}
                  : styles["status-ng"]
              }
            >
              {/* ⑩ OK/NG       */}
              {item.judgement ? "OK" : "NG"}
            </td>
            <td>
              <Link
                {/* ⑪           。URL  id           */}
                to={`/kosu-update/${item.id}`}
```

```
              className={styles["a-collar"]}
            >

            </Link>
          </td>
          <td>
            <Link
              {/* ⑫           。URL  id           */}
              to={`/kosu-delete/${item.id}`}
              className={styles["a-collar"]}
            >

            </Link>
          </td>
        </tr>
      ))}
    </tbody>
  </table>
```

`/kosu-update/42` `:id` `index.tsx`

- `<Link to={` `/kosu-delete/${item.id}` `}>削除</Link>`

> **これが「データ の 」 すべて `data`**
> **（state）から `data`**

8.15 JSX —Pagination と （202〜216 ）

> **▼ このコードがやること:**

```
    <Pagination
      {/* ①               す */}
      currentPage={currentPage}
      {/* ②               す */}
      totalPages={totalPages}
      {/* ③                   す    か          う   */}
      setCurrentPage={setCurrentPage}
      {/* ④                */}
      buttonColor="#0ff"
      {/* ⑤             い     */}
      hoverColor="#0af"
    />
      </TableContainer>
    )}
    </div>
  </>
);
};

{/* ⑥ こ                  */}
export default KosuList;
```

- **`currentPage={currentPage}`** `currentPage` `Pagination`

- `totalPages={totalPages}`
- `setCurrentPage={setCurrentPage}` を props として す `Pagination` `setCurrentPage(3)` の state が わり
- `buttonColor="#0ff" hoverColor="#0af"` `Pagination` `KosuList` `light_blue_button`
- `export default KosuList` `index.tsx` `import KosuList from "./KosuPage/KosuList"`

これで `KosuList.tsx` 215 を み りました。

なフルセット

この1ファイルが めれば、 アプリの はすべて めます

---

9. コード ：Pagination.tsx 71

`KosuList` `Pagination` `frontend/src/Components/Pagination.tsx` propsの「 す 」と「 け る 」がつながる

9.1 import と props （1〜10 ）

```tsx
// ① React
import React from "react";
// ② こ CSS
import styles from "../styles/Components/Pagination.module.css";

// ③ け props
interface PaginationProps {
  //
  currentPage: number;
  //
  totalPages: number;
  // け、 さ い
  setCurrentPage: (page: number) => void;
  //
  buttonColor?: string;
  //
  hoverColor?: string;
}
```

`interface PaginationProps` KosuList

`KosuList`

KosuList（ ）が　す	Pagination（ ）が　ける
`currentPage={currentPage}`	`currentPage`
`totalPages={totalPages}`	`totalPages`
`setCurrentPage={setCurrentPage}`	`setCurrentPage`
`buttonColor="#0ff"`	`buttonColor`　　`#fff`
`hoverColor="#0af"`	`hoverColor`　　`#fff`

9.3 ページ　　の　　4つ（19〜26　　）

> **▼ このブロックがやること:**
>
> `setCurrentPage`

```
// ①　　1
const handleFirstPage = () => setCurrentPage(1);
// ②
const handleLastPage = () => setCurrentPage(totalPages);
// ③
const handleNextPage = () => {
  //　　+1
  if (currentPage < totalPages) setCurrentPage(currentPage + 1);
};
// ④
const handlePreviousPage = () => {
  //　1　　き け　　-1
  if (currentPage > 1) setCurrentPage(currentPage - 1);
};
```

- **`handleFirstPage`**　`setCurrentPage(1)`
- **`handleLastPage`**　`setCurrentPage(totalPages)`
- **`handleNextPage`**　`currentPage < totalPages`　　`+1`
- **`handlePreviousPage`**　`currentPage > 1`　　`-1`

な　　：　　から　け　った `setCurrentPage`
のstateを　かしている　　`KosuList`　　`currentPage`
props（　り）と　　び
し（　り）でデータが

9.4 JSX とCSS　　の　　（28〜67　　）

▼ このブロックがやること:

```
return (
  <div
    className={styles["pagination"]}
    style={{
      // ①   け          CSS
      "--btn-bg-color": buttonColor,
      //
      "--btn-hover-color": hoverColor
    // ②  TypeScript  「CSS    」     え
    } as React.CSSProperties}
  >
    <button
      className={styles["prev-button"]}
      // ③ 1             「    」
      disabled={currentPage === 1}
      //
      onClick={handleFirstPage}
    >
      3
    </button>
    <button
      className={styles["prev-button"]}
      // ④ 1             「  」
      disabled={currentPage === 1}
      onClick={handlePreviousPage}
    >

    </button>
    <span>
      // ⑤ 「2 / 5」     う
      {currentPage} / {totalPages}
    </span>
    <button
      className={styles["next-button"]}
      // ⑥               「  」
      disabled={currentPage === totalPages}
      onClick={handleNextPage}
    >

    </button>
    <button
      className={styles["next-button"]}
      // ⑦               「
```

```
    </div>
  );
```

- `style={{ "--btn-bg-color": buttonColor, ... }}` `#0ff` CSS `Pagination.module.css` `var(--btn-bg-color)` propsで した が、CSSにまで わる
  - `style={{ ... }}` `{ }` `{ }`
- `as React.CSSProperties` `--`
- `disabled={currentPage === 1}` `currentPage` `disabled` `disabled` `true`
- `<span>{currentPage} / {totalPages}</span>` `2 / 5` `{ }`
- `disabled={currentPage === totalPages}`
- `onClick` `setCurrentPage`

9.5 （69〜71 ）

```
};

// こ              KosuList  が import す
export default Pagination;
```

`Pagination.tsx` 71 を み りました。 のstateを たず

`<Pagination .../>` 「propsだけで く ＝どこでも える 」

10.　コード　　：Loading.tsx　42

`frontend/src/Components/Loading.tsx`　　`KosuList`
`useEffect`

10.1 import と props（1〜7　）

```
// React  useEffect  useState
import React, { useEffect, useState } from "react";
// ①
import helmetImage from "../img/helmet.png";
//     CSS
import styles from "../styles/Components/Loading.module.css";

interface LoadingProps {
  // ②   か  「          か」     け
  isLoading: boolean;
}
```

- **`import helmetImage from "../img/helmet.png"`**　　`import`
　　`<img src={helmetImage}>`
- **`isLoading: boolean`**　　`KosuList`　　`<Loading isLoading={loading} />`

10.2 フェードアウト　のstate（9〜10　）

```
// isLoading    け
const Loading: React.FC<LoadingProps> = ({ isLoading }) => {
  //     す  きか。      true
  const [shouldRender, setShouldRender] = useState(true);
```

- `shouldRender`　　**み　みが　わってもすぐ　さず、1かけてフェードアウトさせてから　す**

> **なぜ　　さないの？**　　`isLoading`
> 　　`shouldRender`

10.3 useE ect とクリーンアップ（12〜20 ）

> **▼ このブロックがやること:**

```
useEffect(() => {
  // ①          が
  if (!isLoading) {
    // ② 1           す                  け
    const timeout = setTimeout(() => {
      // ③
      setShouldRender(false);
    //    1000         1    CSS
    }, 1000);
    // ④         け
    return () => clearTimeout(timeout);
  }
// ⑤ isLoadingが
}, [isLoading]);
```

- **`if (!isLoading)`** `isLoading`
- **`const timeout = setTimeout(() => {...}, 1000)`** `setTimeout` `timeout`
- **`setShouldRender(false)`** `null`
- **`return () => clearTimeout(timeout)`** **クリーンアップ** `clearTimeout`

> **なぜタイマーを ける？**
>
> `setShouldRender`

- `}, [isLoading])` `isLoading`

10.4 リターンとJSX（22〜40 ）

```
// ①                    か  い  null
if (!shouldRender) return null;

return (
  <div
    id="loading"
    // ②              fadeOut
    className={`${styles.loading} ${!isLoading ? styles.fadeOut : ""}`}
  >
    <div className={styles.item}>
      <img
        id="loadingImg"
        // ③
        src={helmetImage}
        //
        alt="Loading..."
        //
        className={styles.sway}
      />
    </div>
    // ④ 「Loading...」
    <p className={styles.loadingText}>Loading...</p>
  </div>
);
```

- `if (!shouldRender) return null` `null` も しません
- `className={` ${styles.loading} ${!isLoading ? styles.fadeOut : ""} `}`
  2つのクラスを `styles.loading` `isLoading` `styles.fadeOut`
  - `styles.loading` `styles["loading"]`
- `<img src={helmetImage} ... className={styles.sway}>`
  `sway`
- `<p ...>Loading...</p>`

`Loading.tsx` 42 を み りました。 props（`isLoading`）で と
`useEffect` `setTimeout`

---

11. コード ：TableContainer.tsx 57

`frontend/src/Components/TableContainer.tsx` `KosuList`
「 の さに わせて の の さを し、はみ したらスクロール」
`children`

11.1 import と props （1〜9 ）

```
// ReactNode
import React, { useState, useEffect, useRef, ReactNode } from "react";
import styles from "../styles/Components/TableContainer.module.css";

interface TableContainerProps {
  // ①
  children: ReactNode;
  // ②           ref
  searchBarRef: React.RefObject<HTMLElement | null>;
  // ③     し  ref
  headerRef: React.RefObject<HTMLElement | null>;
  // ④   さ
  heightExpansion?: boolean;
}
```

- **`children: ReactNode`** `children` `<TableContainer>ここ い </TableContainer>` `children` `ReactNode`

> : children（チルドレン） に いた
> `KosuList` `<TableContainer>` `<table>` `<Pagination>`
> `children`

- `searchBarRef`・`headerRef` さ
  `React.RefObject<HTMLElement | null>`
- `heightExpansion?: boolean` `true`
  `KosuList` `false`

11.2 け りと state/ref（11〜18 ）



>

```
useEffect(() => {
  // ①   さ      し       す
  const updateDimensions = () => {
    // ②             さ    け    0
    const searchBarHeight = searchBarRef.current?.offsetHeight || 0;
    // ③     し    さ    け    0
    const headerHeight = headerRef.current?.offsetHeight || 0;

    // ④
    if (heightExpansion) {
      //           - 100px
      setMaxHeight(window.innerHeight - 100);
    // ⑤
    } else {
      setMaxHeight(
        //        -          -     し -     40
        window.innerHeight - searchBarHeight - headerHeight - 40
      );
    }
  };

  // ⑥   ず1
  updateDimensions();

  // ⑦                       す    う
  window.addEventListener("resize", updateDimensions);
  // ⑧       け
  return () => {
    //
    window.removeEventListener("resize", updateDimensions);
  };
// ⑨ こ    が
}, [searchBarRef, headerRef, heightExpansion]);
```

- `const updateDimensions = () => {...}`
- `searchBarRef.current?.offsetHeight || 0` の さ
  `offsetHeight` `?.`
  `|| 0`
-
-
  - `heightExpansion=true` `- 100`
  - `-` `-` し `- 40` から と バーの を いた り

- `updateDimensions()`
- `window.addEventListener("resize", updateDimensions)` `resize` `updateDimensions` を めて も が
- `return () => { window.removeEventListener(...) }` クリーンアップ

- `[searchBarRef, headerRef, heightExpansion]`

11.4 JSX（42〜57 ）

```
return (
  <div
    // ①  ref か
    ref={containerRef}
    className={styles["table-wrapper"]}
    style={{
      // ②  し さ
      maxHeight: `${maxHeight}px`,
      // ③
      overflowY: "auto",
      // ④
      overflowX: "auto",
    }}
  >
    // ⑤ か さ Pagination く
    {children}
  </div>
);
```

- `ref={containerRef}`
- `maxHeight: ${maxHeight}px`` `"450px"`
- `overflowY: "auto", overflowX: "auto"` し・ バーは で、 だけがスクロール
- `{children}` ここが `KosuList` `<TableContainer>...</TableContainer>` `<table>` `<Pagination>` `TableContainer`

`TableContainer.tsx` 57 を み りました。 `children`
`useEffect` `resize`
`Pagination` な

---

12. コード : axios.ts 8

`frontend/src/api/axios.ts` アプリの がサーバーと
すときの `KosuList` `api.get(...)` `Login`
`api.post(...)`

> : axios（アクシオス）
> `api.get(URL)` `api.post(URL, )`

> ▼ このコードがやること:
> `api`

```
// ① axios
import axios from "axios";
// ②
const api = axios.create({
    // ③ URL か
    baseURL: process.env.REACT_APP_API_BASE_URL,
    // ④ Cookie
    withCredentials: true,
    // ⑤ CSRF が Cookie
    xsrfCookieName: "csrftoken",
    // ⑥ CSRF
    xsrfHeaderName: "X-CSRFToken",
});
// ⑦ こ api
export default api;
```

- `import axios from "axios"`

- `const api = axios.create({ ... })` `axios.create` を った のaxios
  `api` `api.get` `api.post`

> : インスタンス（instance）
> `axios.create`

- `baseURL: process.env.REACT_APP_API_BASE_URL` URL
  `api.get("/api/kosu_list/")` `baseURL + "/api/kosu_list/"`
  `process.env.REACT_APP_...`

> : （かんきょうへんすう） `.env`
> コードを えずに
> `REACT_APP_`

- `withCredentials: true` Cookieを に る
  `true`
  ログインを する

> : Cookie（クッキー）／セッション

- `xsrfCookieName` / `xsrfHeaderName` CSRF `csrftoken`
  `X-CSRFToken`

> : CSRF（シーサーフ：Cross-Site Request Forgery）

- `export default api` `api` `KosuList` `Login` `import api from "../api/axios"`

`axios.ts` 8 を み りました。

がうまくいかないときは、まずこのファイルと を う

---

13. ルーティング：index.tsx とSPA

`frontend/src/index.tsx` どのURLでどの を すか（ルーティング） `KosuList` `<Link to="/kosu-update/42">`

13.1 SPA（シングルページアプリケーション）とは

> : SPA（エスピーエー：Single Page Application）
> JavaScriptで だけを し えて

しいHTMLを ごと

じ の で だけ き える
React Router

13.2 アプリの （172〜180 ）

> ▼ このコードがやること: `id="root"`
> `Router`

```
// ①     す
const root = ReactDOM.createRoot(document.getElementById('root') as HTMLElement);

// ②  こ
root.render(
  // ③
  <React.StrictMode>
    // ④
    <Router>
      // ⑤                App
      <App />
    </Router>
  </React.StrictMode>
);
```

- `const root = ReactDOM.createRoot(document.getElementById('root') as HTMLElement)` `public/index.html` `<div id="root"></div>` `as HTMLElement`
- `root.render(...)`
- `<React.StrictMode>`

> : StrictMode（　モード）

- `<Router>` `BrowserRouter` `as Router` これで　んだ `Routes` `Route` `Link` `useNavigate`
- `<App />`

13.3 import　とタイトル　（1〜115　）

ブラウザのタブのタイトル

```
// ①
import { BrowserRouter as Router, Routes, Route, useLocation } from "react-router-dom";
// ②           CSS
import './styles/global.css';
// ③     、
import Login from './MainPage/Login';
//              。§8      し
import KosuList from './KosuPage/KosuList';
// ...         ...
```

- `import { BrowserRouter as Router, Routes, Route, useLocation } from "react-router-dom"` `BrowserRouter` `Router` `as` `Routes` `Route` `useLocation`
- `import './styles/global.css'` `from`
- ～

```
const App: React.FC = () => {
  //       URL
  const location = useLocation();

  React.useEffect(() => {
    //     →
    const routeTitles: { [key: string]: string } = {
      "/login": "          -                  ",
      "/kosu-list": "          -                  ",
      // ...           ...
    };
    //
    document.title = routeTitles[location.pathname] || "                ";
  // URLが
  }, [location]);
```

- `const location = useLocation()`
- `routeTitles` `{ [key: string]: string }`
- `document.title = routeTitles[location.pathname] || "                "` `useEffect` `[location]`

が わるたびにタブ も で わります

13.4 Routes と Route（117〜169 ）

> **▼ このコードがやること:**
>
> Route

```
return (
  {/* ①                */}
  <Routes>
    {/* ② /login     Login     */}
    <Route path="/login" element={<Login />} />
    {/* ③ /                    */}
    <Route path="/" element={<MainMenu />} />
    {/* ④ /kosu-list     KosuList  §8   */}
    <Route path="/kosu-list" element={<KosuList />} />
    {/* ⑤ :id   き      URL */}
    <Route path="/kosu-update/:id" element={<KosuEdit />} />
    {/* ⑥           :id  き */}
    <Route path="/kosu-delete/:id" element={<KosuDelete />} />
    {/* ...   50             ... */}
    {/* ⑦                    */}
    <Route path="/member-update/:employee_no" element={<MemberEdit />} />
  </Routes>
);
```

- **<Routes>** Route **のURLに に する1つ**
- **<Route path="/login" element={<Login />} />** /login <Login /> path element
- **<Route path="/kosu-list" element={<KosuList />} />** /kosu-list KosuList KosuList <nav> <Link to="/kosu-menu">
- **<Route path="/kosu-update/:id" element={<KosuEdit />} />** ここが :id **URLパラメータ** /kosu-update/42 /kosu-update/99

`42` `99` `KosuList` `<Link to={` `}>`

> **：URLパラメータ（`:id`）** `:id` `KosuEdit` `useParams()` `const { id } = useParams()` **1つのルート で、 レコードの に**

- `:employee_no` `:employee_no`

13.5 ルーティングの

`KosuList`

```
①        が <Link to="/kosu-update/42">            §8.14
      ▼
② React RouterがURL   /kosu-update/42                        し  SPA
      ▼
③ <Routes> が、     す  Route <Route path="/kosu-update/:id">
      ▼
④ 🏃 element <KosuEdit />       。:id        42    useParams
      ▼
⑤ KosuEditが id=42          api.get      し、
```

ページ は み みされず `KosuList` `KosuEdit`

---

14. Link と useNavigate — の

`<Router>`

14.1 `<Link>` —クリックで するリンク

> **：Link（リンク）** `<a>` **ページを**

み　みせずSPA　に　　to="　　"

KosuList

```
{/*　　§8.12⑥　*/}
<Link to="/kosu-menu">　MENU</Link>
{/*　。id　　§8.14⑪　*/}
<Link to={`/kosu-update/${item.id}`}>　</Link>
```

- to="/kosu-menu"
- to={`/kosu-update/${item.id}`}　　なURL

なぜ <a href> でなく <Link> ？　<a href="/kosu-menu">
ごと　み　み　　<Link>
<Link>　　<a href>

14.2 useNavigate ―プログラムから

：useNavigate（ユーズナビゲート）　　コードの　で
const navigate = useNavigate()　　navigate("/login")

KosuList　fetchData　　Login

```
//　　§8.5⑩
const navigate = useNavigate();
// ...
//　　§8.7⑬
if (err.response?.status === 401) navigate("/login");
// ...
//　　Login.tsx
if (data.status === "success") navigate("/");
```

- `navigate("/login")` プログラムの で
- `navigate("/")`

> い け: して `<Link>` に じて で
> `useNavigate` `<Link>`
> `useNavigate`

---

15. サイクルの （まとめ ）

`KosuList`

```
【 】
1. /kosu-list → index.tsx <Route> が KosuList §13
2. KosuList state loading=true §8.5
3. loading=true <Loading> §8.11 §10
4. useEffect② §8.9 が → fetchData(1, "", false) §8.7
5. api.get /api/kosu_list/ GET §12 api Cookie CSRF き
6. → setData(...) setTotalPages(...) setLoading(false)
7. state → loading=false → Pagination が §8.14

【「 」 す 】
8. Pagination 「 」→ handleNextPage → setCurrentPage(2) §9.3
※ が、 か さ setCurrentPage §3.2 「 」
9. currentPage が 2 → useEffect② currentPage が §8.9
10. fetchData(2, ...) 2 GET → setData → →
```

「state → （ り：props）」と「 → state （ り： された ）」
`KosuList` `Pagination`

---

16. トラブルシューティング

		?.
Too many re-renders	setState	setState useCallback
map	key	<tr key={item.id}>
	onClick={fn(arg)}	onClick={() => fn(arg)} ⚠️
	x = 1	setX(1) ⚠️
		axios.ts withCredentials: true
<Link>	<a href>	<Link to>
		className={styles[" "]} " "
Cannot read properties of undefined		data?.results

17.

ローカル

stateの を える KosuList.tsx useState<number>(1) currentPage useState<number>(2)

propsで　を　える　`KosuList.tsx`　`<Pagination buttonColor="#0ff" .../>`
`buttonColor="#0f0"`　`Pagination`

　　を　む　`formatTyoku`　　`case 7: return "　　";`　　`tyoku2`
`"7"`

early returnを　　`KosuList.tsx`　`if (loading) return ...`

　　を　　する　`useEffect`　　`currentPage`

map と key　`<tr key={item.id}>`　`key={item.id}`

Linkとaの　い　`<Link to="/kosu-menu">`　`<a href="/kosu-menu">`

　しいコンポーネントを　る　`Components/Hello.tsx`　　`const Hello: React.FC<{name: string}> = ({name}) => <p>こ　　{name} さ　</p>;`　　`export default`
`KosuList`　`<Hello name="　　" />`

---

18. この　のまとめ

- 　　データを　えたら、　は　　から　で　り　される　DOM
- 　　コンポーネント（　）　　まり
- JSX　　`class→className`　`for→htmlFor`　`{ }`　`&&`　　`map`　`key`　　`<> </>`
- props　`Pagination`
- state（`useState`）　　`setX`

- `useEffect` クリーンアップ
- `useRef` `useCallback`

- `KosuList.tsx` 215

- `Pagination.tsx`（propsだけで く ）、`Loading.tsx`（クリーンアップ）、`TableContainer.tsx`
- `axios.ts` 8

- `index.tsx` `Router` `Routes` `Route` `:id` SPA
  `<Link>` `useNavigate`
- 「state→ （ り：props）」と「 →state （ り： された ）」の